Event-Driven Learning Personalization Using Multimodal Analytics and RL

2026 8th International Conference on Computer Science and Technologies in Education (CSTE) | 979-8-3315-6374-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/CSTE69562.2026.11649708

Natalia Axak *
Department of Information Systems and Technologies
Kharkiv National University of Radio Electronics
Kharkiv, Ukraine
nataliia.axak@nure.ua

Maksym Kushnaryov
Department of Information Systems and Technologies
Kharkiv National University of Radio Electronics
Kharkiv, Ukraine
0000-0002-3772-3195

Andrii Tatarnykov
Department of Information Systems and Technologies
Kharkiv National University of Radio Electronics
Kharkiv, Ukraine
0000-0002-1632-8188

Ihor Novoseltsev
Senior Member, IEEE, Computer Science
Independent Researcher
Carlsbad, CA, USA
i.v.novoseltsev@gmail.com

***Abstract*—This paper presents a microservice-and-agent-based architecture for real-time personalization in distributed e-learning systems. The study aims to address the limitations of conventional LMS/MOOC platforms, which predominantly rely on offline analytics and lack coordinated, low-latency adaptation under dynamic learner behavior and fluctuating system load. The objectives of this work are: (i) to design and validate an end-to-end event-driven pipeline for continuous collection and processing of learner behavioral data, (ii) to operationalize a coordinated agent layer that supports adaptive decision-making based on reinforcement learning policies, and (iii) to ensure predictable and conflict-free personalization through formal specification and verification of safety and liveness properties. The proposed architecture integrates microservice decomposition, an event bus for streaming learning events, and an agent layer implementing Q-learning and Proximal Policy Optimization for adaptive control. Learner behavior is modeled using a compact state–action–reward formulation derived from behavioral indicators, with optional enrichment via computer vision–based engagement signals. Formal correctness is achieved by expressing coordination constraints in temporal logics and validating agent interactions through model checking. The resulting framework provides a reproducible and extensible foundation for scalable, low-latency, and formally verified personalization in modern e-learning platforms.**

Keywords—event-driven architecture, reinforcement learning, multi-agent systems, real-time personalization, e-learning

I. Background and Problem Statement of Real-time Learning Personalization in LMS/MOOC Platforms

Distance and blended learning platforms (LMS/MOOC) have become the default infrastructure for delivering courses at scale. However, most deployments still implement personalization as an "offline" or "semi-online" feature: periodic dashboards, end-of-week analytics, rule-based nudges, and static sequencing of materials. These mechanisms can improve reporting, but they rarely support real-time, closed-loop adaptation of learning trajectories while a student is actively interacting with the system.

A. Why Lms/mooc Setups Struggle with Real-time Personalization under Load

There are three structural limitations that make real-time personalization difficult in typical LMS/MOOC architectures.

(1) *Latency of feedback loops*. Many platforms rely on batch processing of logs or scheduled recomputation of indicators. As a result, actionable signals (e.g., disengagement, abnormal test-taking patterns, rapid guessing, or repeated errors on key concepts) are detected late–after the critical moment for intervention has passed. Real-time personalization requires an event-driven loop where evidence is captured and interpreted immediately, and an adaptive decision is issued within strict latency constraints.

(2) *Limited behavioral observability.* Standard LMS telemetry is often dominated by coarse events (page views, clicks, quiz scores) and lacks fine-grained behavioral and engagement cues. Even when richer signals exist, they may be fragmented across modules or external tools. In practice, engagement and integrity-related situations are under-observed, which weakens both personalization quality and reliability of proctoring/monitoring decisions. Multimodal signals (e.g., computer-vision-derived indicators) can increase observability, but they must be integrated into a consistent event stream and privacy-aware processing pipeline.

(3) *Scalability and consistency issues in distributed adaptation*. Personalization at scale is not only an analytics problem; it is a distributed decision-making problem. Under varying system load, thousands of concurrent learners generate bursty event streams. Decisions must remain consistent across multiple services (assessment, content delivery, recommendations, monitoring) and across multiple adaptation mechanisms (rules, agent policies, RL). Typical LMS architectures are modular, but they are not designed for high-frequency event processing with coordinated control actions.

B. Why Coordinated Agents Are Needed

A practical real-time personalization system cannot rely on a single monolithic "recommender" component. Adaptation decisions are distributed across several functional roles: monitoring, feature extraction, risk detection, recommendation

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:14:56 UTC from IEEE Xplore. Restrictions apply.

generation, trajectory correction, and resource-aware scheduling. If these roles act independently, the platform can produce conflicting actions–for example, a recommendation agent increases task difficulty while an engagement agent reduces cognitive load; or integrity detection flags abnormal behavior while a tutoring agent continues to adapt as if the session were normal.

Therefore, personalization requires coordination mechanisms that ensure agents and services make compatible decisions under dynamic conditions (changing learner behavior, incomplete data, and fluctuating compute capacity). This coordination is especially critical when decisions are triggered by streaming behavioral events and must satisfy platform-level properties such as safety (no contradictory actions), liveness (a critical event is eventually handled), and bounded reaction time.

C. Problem Statement

The core problem addressed in this work is the following: How to design a distributed e-learning platform that (i) processes behavioral event streams in real time, (ii) supports multimodal monitoring for more reliable behavioral indicators, and (iii) enables coordinated, conflict-free personalization decisions by multiple agents and services under variable load and resource constraints?

To tackle this problem, we focus on an event-driven microservice-and-agent architecture where behavioral signals are converted into formalized indicators, adaptation decisions are generated by coordinated agents (including RL-based policies), and key interaction properties are controlled to avoid logical conflicts and unpredictable system behavior.

II. Related Work

Learning analytics in LMS/MOOC ecosystems has evolved from descriptive tracking to predictive models and analytics-driven interventions. However, recent reviews show that most approaches remain dashboard-oriented and human-in-the-loop, supporting instructor decisions rather than enabling closed-loop, real-time personalization with explicit stability and safety guarantees under variable load [1]. Studies of AI-powered learning analytics dashboards report increased use of machine learning but also persistent gaps in actionability, evaluation rigor, and operational integration for continuous adaptation [2]. Human-centred learning analytics further highlights the need for transparency, agency, and governance when moving toward automated personalization [3], motivating architectures that combine real-time adaptation with trust and system-level guarantees.

Multi-agent approaches in AIEd provide foundations for distributed personalization and automated pedagogical decision-making. While recent surveys highlight scalability and automation benefits, they also report persistent challenges in interpretability, privacy, and the risk of inconsistent or unsafe interventions when multiple agents act concurrently [4], [5]. These findings underscore the need for explicit cross-agent coordination and decision-consistency mechanisms beyond the mere adoption of multi-agent architectures.

From a systems perspective, real-time personalization relies on event-stream ingestion and low-latency processing of high-volume behavioral data. Surveys of stream processing and runtime adaptation highlight architectures that sustain throughput and bounded latency under fluctuating workloads [6], [7], while empirical studies caution that event-driven decompositions introduce non-trivial overhead and require careful performance engineering [8]. In LMS settings, practical integrations such as Moodle–Kafka pipelines demonstrate near real-time monitoring capabilities but largely stop short of coordinated, policy-driven adaptation with decision guarantees [9]. Although interoperability standards like Caliper and xAPI standardize learning event representation and exchange [10], [11], they do not by themselves ensure safe closed-loop control or consistent multi-agent decisions.

Multimodal learning analytics extends traditional clickstream analysis by incorporating video, audio, and other sensor-derived signals. While recent reviews document rapid growth of MMLA methods and architectures, they also identify persistent challenges related to valid inference, robustness under noise and missing data, and actionable feedback [12], [13]. Conceptual frameworks emphasize grounding MMLA in ethical principles and meaningful interventions rather than purely technical sensing [14], and privacy-focused syntheses warn of practicality and encroachment risks as sensing becomes more intrusive [15]. These findings highlight the need for governance-aware, constraint-driven multimodal pipelines in real-time adaptive learning systems.

Gap and positioning. Despite advances across these areas, a key gap remains: a deployable end-to-end architecture that streams behavioral and multimodal data, supports policy-driven personalization (including RL), and ensures consistent multi-agent decisions under fluctuating load, with formal guarantees where needed. Our prior work addresses microservice-based behavioral personalization [16] and temporal-logic specification and verification of agent interactions [17]. Building on these results, this study targets coordinated, verifiable, real-time personalization at the intersection of systems and AIEd in distributed LMS/MOOC environments.

III. Platform Architecture: Microservices, Event Bus, Agent Layer, and LMS Integration

A. Design Goals

The platform is designed to enable real-time personalization under fluctuating system load while maintaining modularity and consistent decision-making across distributed components. To achieve this, the architecture is guided by four core principles: microservice decomposition to support independent scaling and evolution; event-driven integration to ensure low-latency data ingestion and loose coupling between producers and consumers; an agent-based decision layer for coordinated adaptive control; and alignment with educational event standards (e.g., Caliper/xAPI) to minimize integration overhead with existing LMS ecosystems [10], [11].

B. High-level Architecture Overview

Fig. 1 shows the end-to-end dataflow: the LMS (or MOOC front-end) emits learning events to an event bus, where microservices perform validation, enrichment, feature extraction, and online analytics to build a learner state. This state is consumed by an agent layer that generates adaptation actions

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:14:56 UTC from IEEE Xplore. Restrictions apply.

(recommendations, trajectory updates, alerts) and delivers them back to the LMS via integration adapters.

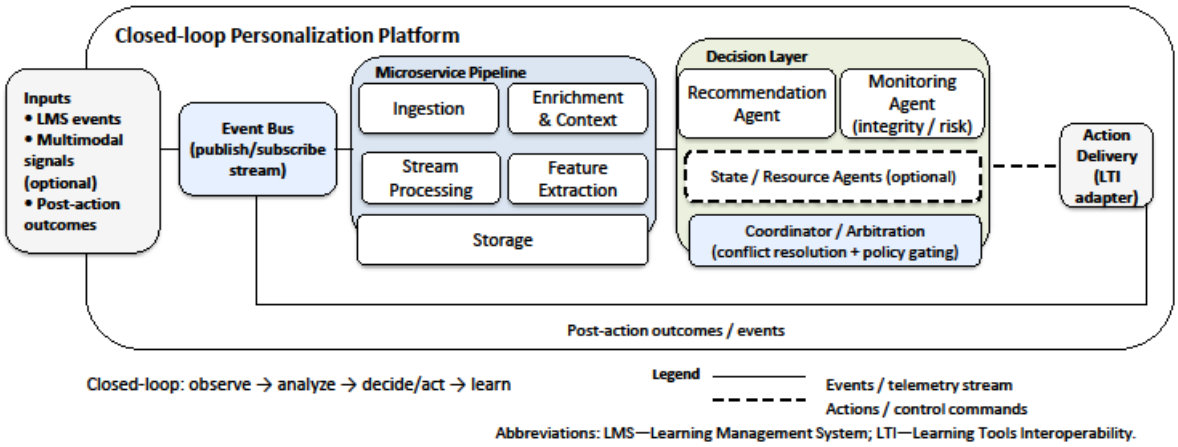


Fig. 1. Event-driven closed-loop architecture for learning personalization. Solid arrows denote event/telemetry streams (LMS learning events, multimodal signals, and post-action outcomes) delivered via the event bus to the microservice pipeline and state construction. Dashed arrows denote control actions produced by the coordinated decision layer (recommendation + monitoring with arbitration) and delivered through the LMS adapter. Abbreviations: LMS – Learning Management System; LTI – Learning Tools Interoperability.

Fig. 1 provides an implementation-oriented view, emphasizing that the pipeline stages are deployed as independently scalable microservices. Event flow (closed-loop): (i) Observe: LMS → event bus (raw events); (ii) Analyze: processing microservices → indicators and learner state; (iii) Decide: coordinated agents (rules/RL) → actions; (iv) Act: LMS integration adapter → UI/next activity/restrictions; (v) Learn: outcomes and feedback → event bus (post-action events). In Fig. 1, solid arrows denote event/telemetry streams, whereas dashed arrows denote control actions/commands returned to the LMS.

C. Microservice Layer

The microservice layer realizes the Observe – Analyze stages as five independently scalable services: (i) Ingestion (schema validation, event-time stamping, privacy filters), (ii) Enrichment & Context (course/session mapping, consent and governance flags), (iii) Stream Processing (event-time windowed operators for trends and anomalies), (iv) Feature Extraction (online indicators consumed by agents), and (v) Storage (immutable event log plus materialized views for dashboards and model updates). Optional sensing (e.g., CV) publishes derived features to the same canonical stream after temporal alignment. This decomposition isolates faults and enables independent scaling of "hot" stages (ingestion/stream/feature extraction) from "cold" stages (storage/retrospective analytics).

D. Event Bus and Streaming Substrate

The event bus decouples producers (LMS and optional sensing services) from consumers (processing microservices and agents) and carries events in a platform-level canonical schema that is stable across pipeline stages (Fig. 1). LMS-originated events are mapped to this schema at Ingestion; identity resolution, course context, and consent/governance metadata are attached during Enrichment & Context, preserving interoperability with Caliper/xAPI while keeping internal processing schema-stable [10], [11].

Events are processed as event-time streams with windowed, stateful operators to handle out-of-order arrivals and bursty workloads [6], [7]. Because event-driven decomposition may introduce serialization and network overheads [8], the prototype explicitly manages latency budgets (P50/P95), applies bounded queues/backpressure, and enforces idempotent processing via unique event identifiers; raw events and derived indicators are persisted for auditability and post-hoc analysis.

E. Agent Layer: Coordinated Decision-making

The agent layer implements closed-loop adaptive control by consuming behavioral states from microservices and issuing adaptation actions. Agents are role-based: a state agent maintains compact learner/session representations; a recommendation agent selects pedagogical actions (sequencing, remediation, pacing); a monitoring agent detects critical situations and constrains adaptation when necessary; and a resource-aware agent adjusts analysis depth or model complexity under load.

Coordination and consistency. To avoid conflicting interventions, the platform employs a coordination mechanism based on a shared decision contract and policy arbitration that enforces global constraints on agent actions, optionally strengthened by offline verification of safety and liveness properties (Section V).

This ensures that the platform's behavior remains predictable when multiple agents act on partially observed and rapidly changing streams.

F. LMS Integration Patterns

Integration with LMS/MOOC is handled by an LMS adapter that supports both outbound events and inbound actions:

- Outbound (LMS → platform): plugin/webhook that publishes learning events to the bus (or to an HTTP gateway that forwards to the bus). A Kafka-based

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:14:56 UTC from IEEE Xplore. Restrictions apply.

Moodle plugin demonstrates feasibility of this integration approach for near real-time monitoring [9].

- Inbound (platform → LMS): action delivery via REST APIs, LTI-based services, or plugin callbacks. Actions include: personalized recommendations, next-activity selection, pacing adjustments, and instructor-facing alerts.

The adapter also provides consent and governance hooks (e.g., enabling/disabling multimodal sensing per course) and maintains mappings between LMS objects (course/quiz/item IDs) and platform identifiers.

The proposed architecture combines (i) microservice modularity for scalable processing, (ii) event-driven streaming for low-latency analytics, (iii) a coordinated agent layer for consistent adaptive decisions, and (iv) LMS adapters grounded in education event standards where possible [10], [11]. This design directly targets the gap between dashboard-oriented analytics and real-time, closed-loop personalization under realistic workload variability.

IV. Behavior Modeling and RL Policy

A. Learner State Representation

Personalization decisions rely on a compact learner state derived from streaming behavioral indicators produced by analytics microservices. The state is updated online in event time and combines short-term dynamics with stable contextual features, including performance, engagement, risk signals, and task context. This representation balances reactivity to recent behavior with stability against over-adaptation, enabling low-latency decision-making under variable load. Formally, the learner state at decision time t is represented as a d-dimensional vector $s_t \in \mathbb{R}^d$, where d depends on the enabled feature groups in Table I (performance, engagement, risk/monitoring, context, and optional CV). For the simulation-based evaluation reported in "Prototype Evaluation (Brief)", we instantiate a compact discrete state with d=3 components (skill level, engagement level, and recent correctness streak), yielding |S|=45 states.

Table I summarizes the state representation, action space, and reward components used by the proposed RL-based personalization framework.

TABLE I. Summary of State, Action, and Reward Components

Component	Category	Description	Source
State	Performance	Accuracy, error streaks, response time statistics	LMS events
	Engagement	Activity frequency, navigation entropy, idle intervals	LMS logs
	Risk/Monitoring	Anomaly score, integrity flags	Monitoring agent
	Context	Course ID, task difficulty, assessment type	LMS metadata
	(Optional) CV	Attention persistence, gaze variability (aggregated)	CV module
Action	Pedagogical	Content recommendation, remediation, pacing adjustment	LMS adapter
	Monitoring	Alert generation, integrity checks	Monitoring agent
	Control	Freeze/limit adaptation under instability	Orchestration agent
Reward	Positive	Performance improvement, sustained engagement	Post-action events
	Negative	Integrity violations, disengagement signals	Monitoring outcomes
	Regularization	Over-intervention penalty, policy oscillation	Policy history

B. Action Space

The action space represents pedagogically meaningful interventions that are delivered through LMS integration adapters. These actions include content recommendations (remedial or advanced), pacing and task sequencing adjustments, monitoring or integrity alerts, and temporary suspension of adaptation under unstable or suspicious conditions. To avoid conflicting interventions, all actions are governed by a shared decision contract (Section III-E) that enforces priorities and mutual exclusions across agents.

C. Reward Design

The reward function captures short-term learning outcomes while penalizing unstable or intrusive behavior by combining performance and engagement gains with penalties for integrity violations, excessive interventions, and abrupt policy changes. Rewards are computed from post-action events published to the event bus, thereby closing the Observe–Analyze–Decide–Act–Learn loop.

D. RL Policies: Q-learning and PPO (Brief)

The platform supports two reinforcement learning strategies. Q-learning is used as a lightweight and interpretable baseline for discrete state–action spaces with minimal computational overhead. Proximal Policy Optimization (PPO) targets richer state representations and more complex policies, offering improved stability under non-stationary behavior and delayed rewards. Both approaches consume streaming state updates; training can be performed either online (incremental updates) or offline (batch/trace-driven), depending on deployment constraints.

Reproducibility details: (i) Policy update frequency: Q-learning applies a TD(0) update after each decision step (event-triggered), while PPO performs mini-batch updates after collecting fixed-length rollouts. (ii) Learning mode: the results reported in "Prototype Evaluation (Brief)" use offline training for 5,000 simulated episodes followed by evaluation of a frozen policy on 1,000 episodes (no learning during evaluation). (iii) Exploration-exploitation: Q-learning uses ε-greedy action selection with exponential decay from ε_{start}=0.30 to ε_{end}=0.05 (time constant τ=1500 episodes); PPO relies on stochastic policy sampling with entropy regularization.

To balance learning effectiveness, engagement stability, and intervention safety, the reward at time step t is defined as a weighted sum of normalized indicators:

$$r_t = \propto \cdot \Delta P_t - \beta \cdot \Delta E_t - \gamma \cdot \Delta C_t - \delta \cdot O_t \quad (1)$$

where: ΔP_t denotes the improvement in learner performance (e.g., accuracy gain or error reduction), ΔE_t captures changes in engagement indicators (e.g., activity regularity, reduced idle time), C_t represents constraint violations (e.g., integrity

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:14:56 UTC from IEEE Xplore. Restrictions apply.

warnings, abnormal behavior), O_t penalizes over-intervention (e.g., frequent or rapidly changing recommendations), α, β, γ, δ are weighting coefficients reflecting pedagogical priorities.

The reward is computed from post-action events published to the event bus, ensuring consistency with the closed-loop adaptation cycle and enabling delayed reward attribution for longer learning sequences.

E. Integration of Computer Vision (CV) Signals

The computer vision (CV) module is integrated as an optional sensing service that augments the behavioral state with non-intrusive engagement proxies, such as attention persistence, gaze dynamics, and head pose variability. To ensure robustness and privacy, CV outputs are not used directly for decision-making but are instead transformed into derived indicators and temporally aligned with LMS events. Under high system load or restricted user consent, the agent layer can gracefully degrade by excluding CV-derived features and relying solely on log-based indicators.

This design enables multimodal enrichment while preserving stability, explainability, and governance constraints. Learner behavior is modeled as a state–action–reward process and RL policies (Q-learning/PPO) operate on streaming state updates under the shared decision contract (Section III-E).

F. Prototype Evaluation (Brief)

We implemented a minimal dockerized prototype of the pipeline in Fig. 1 (ingestion → stream/feature extraction → coordinated agents → LMS action stub) and instrumented timestamps at each stage. Table II reports end-to-end latency (P50/P95), defined as the time from event ingestion to LMS action-stub acknowledgement, and throughput under synthetic concurrent load (R≈1 ev/s per user) for U={10, 50, 100}: P50 increases from 12.34 ms to 98.76 ms and P95 from 25.67 ms to 287.33 ms while throughput scales to 95.35 ev/s. In a simulation-based evaluation, RL (Q-learning) increases mean cumulative reward by 23.0% (18.24±4.68 vs 14.83±5.91) relative to a rule-based baseline, reduces disengagement/dropout from 0.097 to 0.063 (−0.034, 35.1% relative), and lowers mean interventions per session from 9.12 to 7.85. To validate coordination in practice, a two-agent decision simulation with mutual exclusions between pedagogical recommendations and exclusive monitoring actions (proctoring_alert/lock_exam) yields 45.5–46.7 detected conflicts per 1,000 decisions before arbitration and 0 executed conflicts per 1,000 decisions after arbitration (overrides equal to detected conflicts). Finally, SPIN model checking of the abstracted conflict–arbitration model explores 258 stored states / 4,576 transitions (0.01 s, 3.456 MB) for STEPS=10 and 132,156 / 4,405,200 (3.17 s, 18.432 MB) for STEPS=100, with no assertion violations (errors=0; 0 counterexamples) (Table III).

TABLE II. PROTOTYPE EVALUATION SUMMARY

Load (U, ev/s)	Latency P50/P95 (ms)	Thpt (ev/s)	ΔR (%)
10, 10	12.34 / 25.67	9.82	23.0
50, 50	45.21 / 112.45	49.38	23.0
100, 100	98.76 / 287.33	95.35	23.0

TABLE III. SPIN MODEL-CHECKING SUMMARY (CONFLICT–ARBITRATION)

Step	Stored states	Trans. (#)	Time (s)	Mem (MB)	Err / CE
10	258	4,576	0.01	3.456	0
100	132,156	4,405,200	3.17	18.432	0

V. FORMALIZATION AND VERIFICATION

A. Motivation and Scope

Given the distributed and adaptive nature of the proposed platform, correctness cannot be ensured solely by empirical testing. Therefore, we complement prototype benchmarking (Section IV-F) with formal specification and verification to guarantee that critical properties hold under concurrent agent execution, asynchronous event delivery, and fluctuating workload. The focus is placed on safety and liveness properties that are essential for trustworthy personalization in educational settings.

B. Formal System Model

The platform is modeled as a set of interacting components $\mathrm{C} = \{\mathrm{S}_i, \mathrm{A}_j, \mathrm{B}\}$ where: S_i denote microservices (ingestion, analytics, storage), A_j denote agents (state, recommendation, monitoring, orchestration), B denotes the event bus mediating asynchronous communication.

Agent behavior is abstracted as a finite-state transition system, where transitions are triggered by incoming events or derived indicators. Inter-agent coordination and constraints are expressed at the level of shared actions and guards, reflecting the decision contract described in Section III-E.

C. Safety Properties

Safety properties specify that undesirable states are never reached. Representative safety requirements include: (i) No conflicting actions: two mutually exclusive interventions are never applied simultaneously; (ii) Integrity preservation: if an integrity violation is detected, adaptive interventions are suspended; (iii) Bounded intervention rate: the number of adaptations within a time window does not exceed a predefined threshold.

Such properties are naturally expressed using Linear Temporal Logic (LTL). For example, a bounded intervention constraint can be specified as: "Globally, if the intervention counter exceeds the limit, then no new adaptation action is issued."

Illustrative verified safety property (no conflicting actions). Let ped denote a pedagogical recommendation (hint/remediate/advance) and exclMon denote an exclusive monitoring action (proctoring_alert/lock_exam). The constraint is expressed as the LTL safety property: `φ1 = G ¬(ped ∧ exclMon.`

In the Promela model, this is enforced at each arbitration step via: `assert(!(ped && exclMon));`

D. Liveness Properties

Liveness properties guarantee that the system eventually makes progress and does not remain indefinitely in an inactive

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:14:56 UTC from IEEE Xplore. Restrictions apply.

or blocked state. In adaptive learning scenarios, such properties require that valid learning events eventually trigger a decision or state update, sustained disengagement leads to a corrective intervention, and adaptation can be resumed after a temporary suspension once normal behavior is restored. These requirements are specified using Linear Temporal Logic (LTL) or Alternating-time Temporal Logic (ATL) when explicit modeling of agent strategies and choices is required.

E. Verification Approach

Formal specifications are translated into models suitable for model checking (e.g., SPIN/Promela). Verification is performed offline on abstracted system models that capture concurrency and synchronization behavior while remaining independent of concrete data values. This approach enables early detection of logical conflicts between agents, validation of coordination mechanisms under worst-case interleavings, and increased confidence in predictable system behavior prior to deployment. Formal verification complements runtime monitoring by addressing design-level correctness rather than replacing empirical evaluation. Representative state-space statistics are summarized in Table III.

F. Discussion

By combining formal specification with microservice-and-agent architecture, the platform reduces the risk of unstable or contradictory adaptive behavior. Importantly, verification targets the decision logic and coordination layer, remaining independent of the specific learning model (rule-based or RL-driven), which enables incremental evolution of personalization strategies.

VI. Conclusion

This study addressed three measurable objectives, establishing a clear correspondence between the proposed design and the obtained results. First, we developed an end-to-end microservice-based, event-driven architecture for real-time learning personalization, enabling low-latency processing, modular scaling, and robust operation under fluctuating workloads. Second, learner behavior was formalized as a state–action–reward process, and adaptive personalization was implemented using Q-learning and PPO within a coordinated agent layer. This approach supports adaptation to non-stationary learner dynamics while preventing conflicting or excessive interventions through explicit coordination mechanisms. Third, we demonstrated that formal specification and verification of safety and liveness properties complement empirical evaluation by identifying design-level coordination conflicts and unsafe behaviors prior to deployment.

The resulting platform provides a reproducible foundation for scalable, verifiable, and real-time personalization in LMS/MOOC environments. Future work will extend the evaluation to large-scale LMS pilots and deepen the integration of formal guarantees with online monitoring.

References

[1] Z. Pan, L. Biegley, A. Taylor, and H. Zheng, “A systematic review of learning analytics–incorporated instructional interventions on learning management systems,” Journal of Learning Analytics, vol. 11, no. 2, pp. 52–72, 2024, doi: 10.18608/jla.2023.8093.

[2] L. Cabral, R. Pinto, and G. Gonçalves, “AI-powered learning analytics dashboards: a systematic review of applications, techniques, and research gaps,” Discover Education, vol. 4, art. 525, 2025, doi: 10.1007/s44217-025-00964-y.

[3] S. Buckingham Shum, R. Martínez-Maldonado, Y. Dimitriadis, and P. Santos, “Human-centred learning analytics: 2019–24,” British Journal of Educational Technology, vol. 55, no. 3, pp. 755–768, 2024, doi: 10.1111/bjet.13442.

[4] R. Zhang, “Multi-agent systems for learning assessment in education: a comprehensive survey,” in Proc. EKI 2025, pp. 388–391, 2025, doi: 10.1145/3765325.3765390.

[5] C. Dahal, J. Chen, M. Ye, and Z. Xiong, “Multi-agent systems in education: a survey from the trustworthiness perspective,” TechRxiv, 2025, doi: 10.36227/techrxiv.176704820.01980102/v1.

[6] M. Fragkoulis, P. Carbone, V. Kalavri, and A. Katsifodimos, “A survey on the evolution of stream processing systems,” The VLDB Journal, vol. 33, no. 2, pp. 507–541, 2024, doi: 10.1007/s00778-023-00819-8.

[7] V. Cardellini, F. Lo Presti, M. Nardelli, and G. R. Russo, “Runtime adaptation of data stream processing systems: the state of the art,” ACM Computing Surveys, vol. 54, no. 11s, art. 237, pp. 1–36, 2022, doi: 10.1145/3514496.

[8] H. Cabane and K. Farias, “On the impact of event-driven architecture on performance: an exploratory study,” Future Generation Computer Systems, vol. 153, pp. 52–69, 2024, doi: 10.1016/j.future.2023.10.021.

[9] R. K. Chamoun, W. Wazen, and M. Gharib, “Design and implementation of a real-time web infrastructure for student monitoring: a Kafka-based plugin for Moodle,” in Proc. WEBIST 2025, pp. 205–212, 2025, doi: 10.5220/0013753200003985.

[10] 1EdTech Consortium, Inc., “Caliper analytics specification,” ver. 1.2, Final Release, Mar. 27, 2020. [Online]. Available: https://www.imsglobal.org/spec/caliper/v1p2. Accessed: Jan. 9, 2026.

[11] IEEE Standards Association, “IEEE standard for learning technology–JavaScript object notation (JSON) data model format and representational state transfer (RESTful) web service for learner experience data tracking and access,” IEEE Std 9274.1.1-2023, Oct. 6, 2023. [Online]. Available: https://standards.ieee.org/ieee/9274.1.1/7321/. Accessed: Jan. 9, 2026.

[12] H. Ouhaichi, D. Spikol, and B. Vogel, “Research trends in multimodal learning analytics: a systematic mapping study,” Computers and Education: Artificial Intelligence, vol. 4, art. 100136, 2023, doi: 10.1016/j.caeai.2023.100136.

[13] S. Mu, M. Cui, and X. Huang, “Multimodal data fusion in learning analytics: a systematic review,” Sensors, vol. 20, no. 23, art. 6856, 2020, doi: 10.3390/s20236856.

[14] M. Worsley, R. Martinez-Maldonado, and C. D’Angelo, “A new era in multimodal learning analytics: twelve core commitments to ground and grow MMLA,” Journal of Learning Analytics, vol. 8, no. 3, pp. 10–27, 2021, doi: 10.18608/jla.2021.7361.

[15] P. Prinsloo, S. Slade, and M. Khalil, “Multimodal learning analytics–In-between student privacy and encroachment: a systematic review,” British Journal of Educational Technology, vol. 54, no. 6, pp. 1566–1586, 2023, doi: 10.1111/bjet.13373.

[16] N. Axak, M. Kushnaryov, and A. Tatarnykov, “Microservice architecture of the learning personalization system based on the analysis of students’ behavioral data,” CEUR Workshop Proceedings, vol. 3955, pp. 14–26, 2024. [Online]. Available: https://ceur-ws.org/Vol-3955/. Accessed: Jan. 9, 2026.

[17] N. Axak and A. Tatarnykov, “Formalization and verification of agent learning platform using temporal logics,” Information Technology: Computer Science, Software Engineering and Cyber Security, no. 3, pp. 3–13, 2025, doi: 10.32782/IT/2025-3-1 (in Ukrainian).

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:14:56 UTC from IEEE Xplore. Restrictions apply.